

Vulnerability Management

Owner: Mark Stolte, Founder — markstolte02@gmail.com

Last reviewed: 2026-07-05

Stage: Pre-launch / active development

Purpose

Describes how Penny Pilot finds out about, prioritizes, and fixes vulnerabilities in its own code, its dependencies, and the platforms it runs on.

Sources of vulnerability information

Source	What it covers	Current status
GitHub Dependabot alerts	Known-vulnerable npm dependencies	Enabled on <code>Mstolte02/penny-pilot</code> as of 2026-07-05.
<code>npm audit</code>	Known-vulnerable npm dependencies	Run ad hoc before dependency upgrades; not yet scheduled.
Expo SDK release notes / advisories	Vulnerabilities in the Expo runtime and first-party modules	Reviewed when upgrading the Expo SDK version.
Supabase status page / security advisories	Vulnerabilities in Supabase-hosted infrastructure	Subscribed informally; Supabase is responsible for patching its own platform.
Plaid security bulletins / API changelog	Vulnerabilities or breaking changes in the Plaid integration	Reviewed when Plaid API behavior changes affect <code>supabase/functions/plaid-*</code> .
Apple/Google platform security bulletins	OS-level or store-policy security requirements	Reviewed at each app store submission.

Immediate action item from this review

Dependabot security alerts were enabled 2026-07-05 as part of this review (a one-time GitHub setting, no code change required). Dependabot *version updates* (automated PRs that bump vulnerable dependencies) are not yet enabled and remain a follow-up item.

Triage and remediation targets

Given the current team size, these are targets rather than SLAs enforced by tooling:

Severity	Target remediation	Notes
Critical (actively exploited, or directly exposes Restricted-tier data per Data Classification & Handling)	As soon as possible, same day if feasible	E.g., a vulnerability in the Plaid token encryption path or Supabase Auth flow.
High	Within 7 days	E.g., a vulnerable dependency with a known exploit but no current exposure path in this app.
Medium	Within 30 days, bundled with normal development	—
Low	Addressed opportunistically, e.g., during a routine dependency bump	—

Process

1. A vulnerability is identified (via Dependabot once enabled, `npm audit`, a vendor bulletin, or manual discovery).
2. It's assessed for whether the vulnerable code path is actually reachable in Penny Pilot's usage (many npm advisories affect code paths this app doesn't exercise).
3. If it needs a fix, the dependency is upgraded (or the vulnerable pattern removed) on a branch, run through the checks in [Secure Software Development Lifecycle](#), and merged.
4. If a fix isn't immediately available (e.g., waiting on an upstream release), the exposure is documented as an accepted, time-boxed risk per the [Information Security Policy](#).

What's not yet in place

- No automated dependency-update PRs (Dependabot version updates or Renovate) — planned once Dependabot alerts are enabled.
- No penetration test or third-party security audit has been performed. This is appropriate to disclose honestly for a pre-launch app, and is planned before Penny Pilot moves real users onto production Plaid credentials at scale.
- No public responsible-disclosure process (e.g., a `security.txt`) exists yet. Until one is published, the contact for any vulnerability report is markstolte02@gmail.com.